Министерство науки и высшего образования РФ
Федеральное государственное автономное
образовательное учреждение высшего образования
«СИБИРСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт космических и информационных технологий

Прикладная информатика

КУРСОВАЯ РАБОТА

ИС «Учет внутриофисных расходов»
тема

Преподаватель _____ В.С. Васильев
подпись, дата

Студент КИ23-20Б, 032211387 _____ А.А. Полягошко
номер группы, зачетной книжки подпись, дата

Красноярск 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 Спецификация требований 4
1.1 Описание бизнес-процесса 4
1.2 Требования к системе 6
1.3 Форматы сущностей 11
2 Проектирование 13
2.1 Прецедент «Создать расход» 13
2.2 Прецедент «Обработать расходы» 14
2.3 Прецедент «Оформить расходы» 15
2.4 Диаграмма классов 16
3 Разработка и тестирование 18
3.1 Работа с данными 18
3.2 Учетные данные 18
3.3 Изменение структуры базы данных 18
3.4 Сборка программы 18
3.5 Тестирование 19
ЗАКЛЮЧЕНИЕ 20
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 21
ПРИЛОЖЕНИЕ А 22

ВВЕДЕНИЕ

В данной курсовой работе разрабатывается информационная система для учета внутриофисных расходов. В современных условиях эффективное управление расходами внутри организации является важной задачей, требующей четкого контроля и учета всех финансовых операций. Система должна предоставлять возможность фиксировать расходы, классифицировать их по категориям, а также анализировать данные для оптимизации бюджета.

Целью является разработка десктопного приложения «Учет внутриофисных расходов», позволяющее автоматизировать процесс учета расходов.

1 Спецификация требований

1.1 Описание бизнес-процесса

Главная задача проекта — автоматизировать процесс подачи, согласования и оплаты заявок на расходы, а также обеспечить контроль и отчетность финансовых операций.

Приложение должно поддерживать работу четырех групп пользователей: администраторов, начальников, бухгалтеров и рядовых сотрудников. Сотрудники могут создавать заявки на погашение расходов, начальники — утверждать или отклонять их. Бухгалтер оплачивает одобренные заявки, формирует отчеты за выбранные периоды и распределяет остатки средств между отделами или переносит их на следующий месяц. Администратор управляет структурой организации: создает отделы, назначает начальников и сотрудников.

Более подробное описание жизненного цикла основной последовательности действий в программе изображено на диаграмме состояний (рис. 1.1).

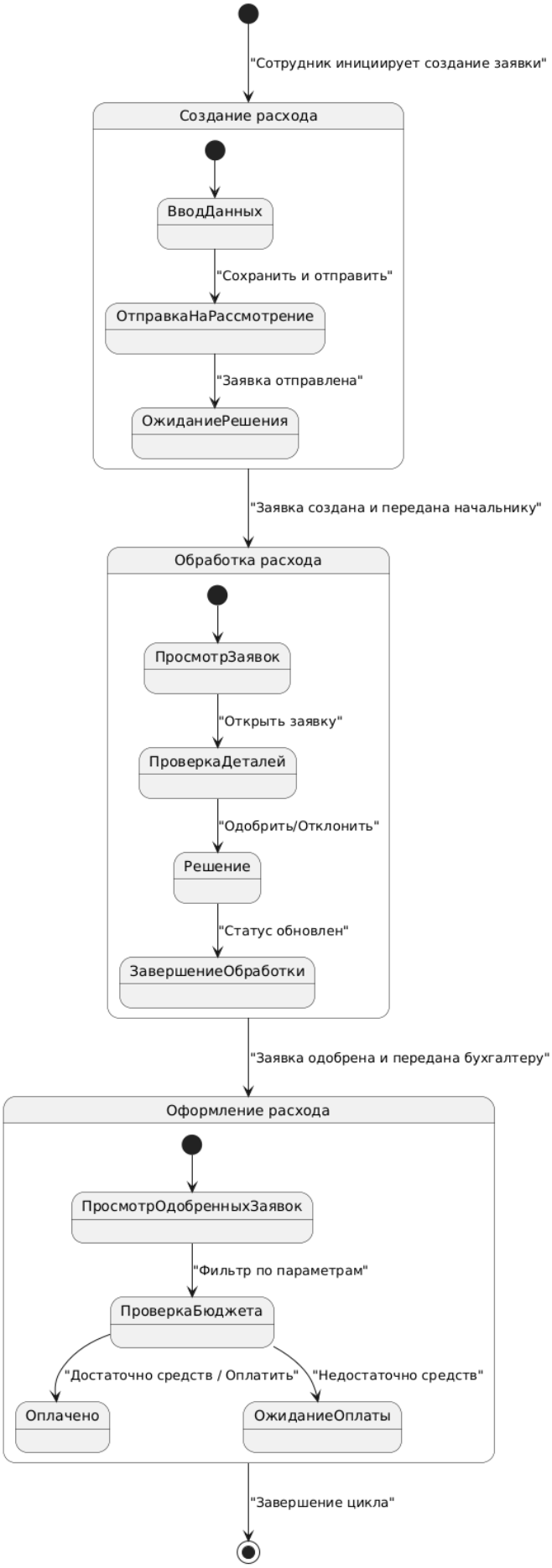

Рисунок 1.1 – Диаграмма состояний

1.2 Требования к системе

На рисунке 1.2 выделены прецеденты для администратора, начальника, бухгалтера и сотрудника.

Рисунок 1.2 – Диаграмма прецедентов

1) Название прецедента: создать расход

Предусловие: после авторизации у сотрудника открыто окно «Главная страница» (рисунок 1.3), на котором он нажимает на кнопку «Создать расход».

Основная последовательность:

1. Сотрудник вводит необходимые данные и нажимает на кнопку «Сохранить и отправить на рассмотрение» (рисунок 1.4).
2. Система формирует заявку и отправляет ее на рассмотрение начальника.
3. Сотрудник получает уведомление об успешном создании заявки.

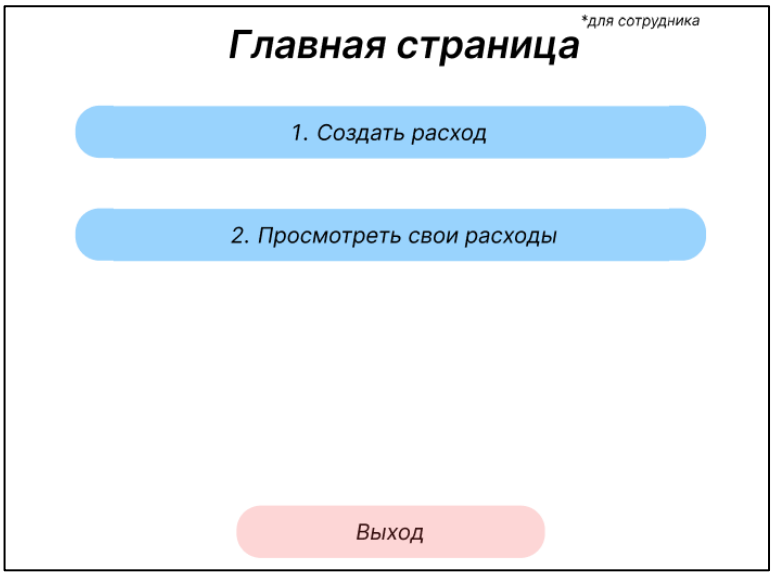

Рисунок 1.3 – Окно «Главная страница»

Рисунок 1.4 – Окно «Создать расход»

2) Название прецедента: обработать расходы

Предусловие: после авторизации у начальника открыто окно «Главная страница», на котором он нажимает на кнопку «Обработать расходы».

Основная последовательность:

1. Система отображает начальнику список всех заявок на рассмотрение (рис. 1.4).
2. Начальник открывает каждую заявку для проверки деталей (рис. 1.5).
3. Начальник принимает решение: одобрить или отклонить расход.
4. Система сохраняет статус заявки (одобрено/отклонено).

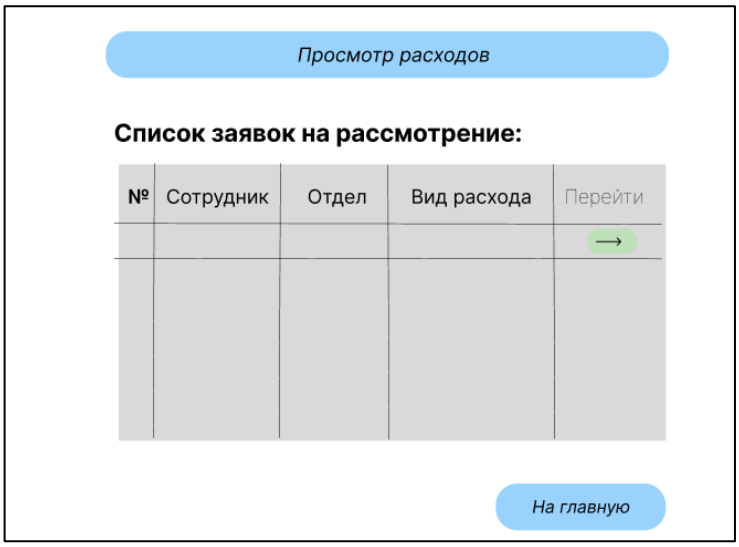

Рисунок 1.5 – Окно «Просмотр расходов»

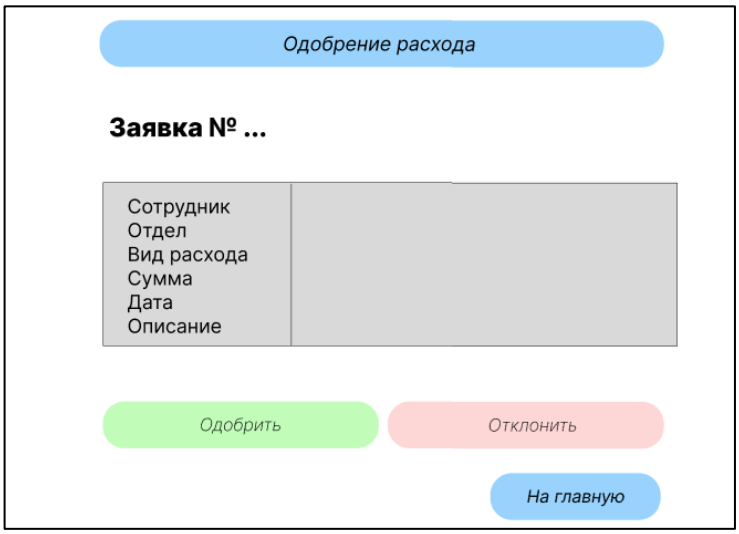

Рисунок 1.6 – Окно «Одобрение расхода»

3) Название прецедента: оформить расходы

Предусловие: после авторизации у бухгалтера открыто окно «Главная страница», на котором он нажимает на кнопку «Просмотреть/оформить расходы».

Основная последовательность:

1. Система отображает бухгалтеру список одобренных заявок, ожидающих оплаты, с возможностью отфильтровать по отделам/видам расходов/дате.
2. Если средств в бюджете достаточно, бухгалтер проводит оплату, нажав на кнопку «Оплатить» в столбце «Статус» (рис. 1.6).
3. Система фиксирует оплату и обновляет статус заявки на «Оплачено».

Альтернативная последовательность:

1. Если бюджет недостаточен, система уведомляет бухгалтера о невозможности завершить оплату.
2. Статус заявки остается «Оплатить».

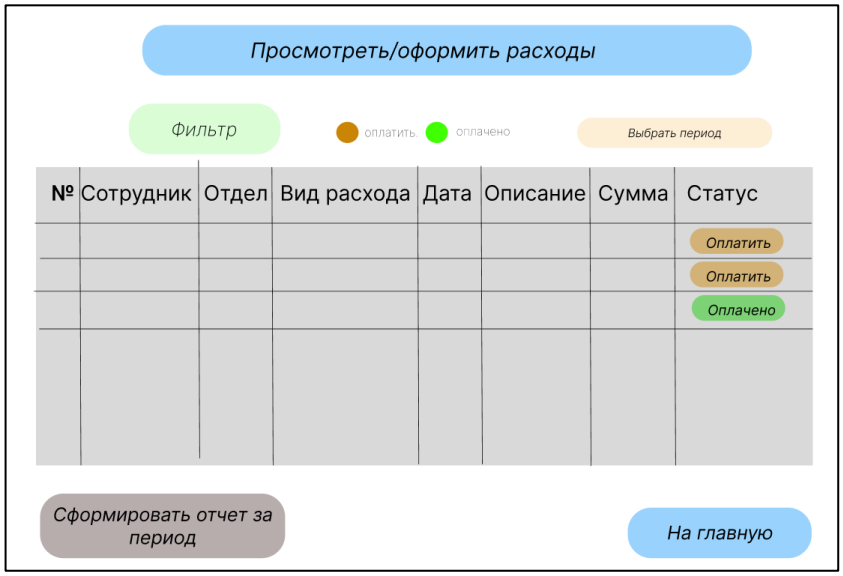

Рисунок 1.7 – Окно «Просмотреть/оформить расходы»

4) Название прецедента: создать отдел

Предусловие: после авторизации у администратора открыто окно «Главная страница», на котором он нажимает на кнопку «Создать отдел».

Основная последовательность:

1. Система выводит администратору форму для заполнения, в которой запрашивает данные нового отдела (рис. 1.7).
2. Администратор дает название отделу, назначает начальника и прикрепляет сотрудников.
3. Нажимает на кнопку «Создать».
4. Система формирует отдел и уведомляет администратора о его успешном создании.

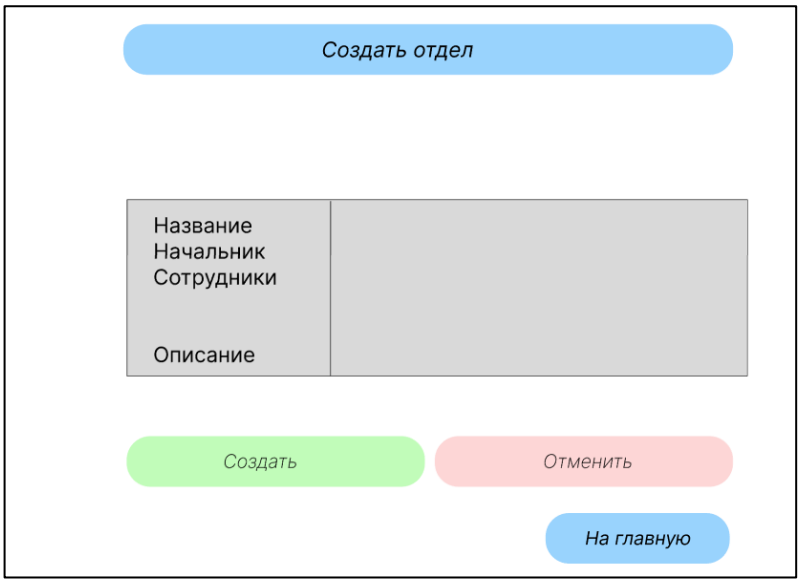

Рисунок 1.8 – Окно «Создать отдел»

На карте диалоговых окон подробно показано использование пользовательского интерфейса (рис. 1.8).

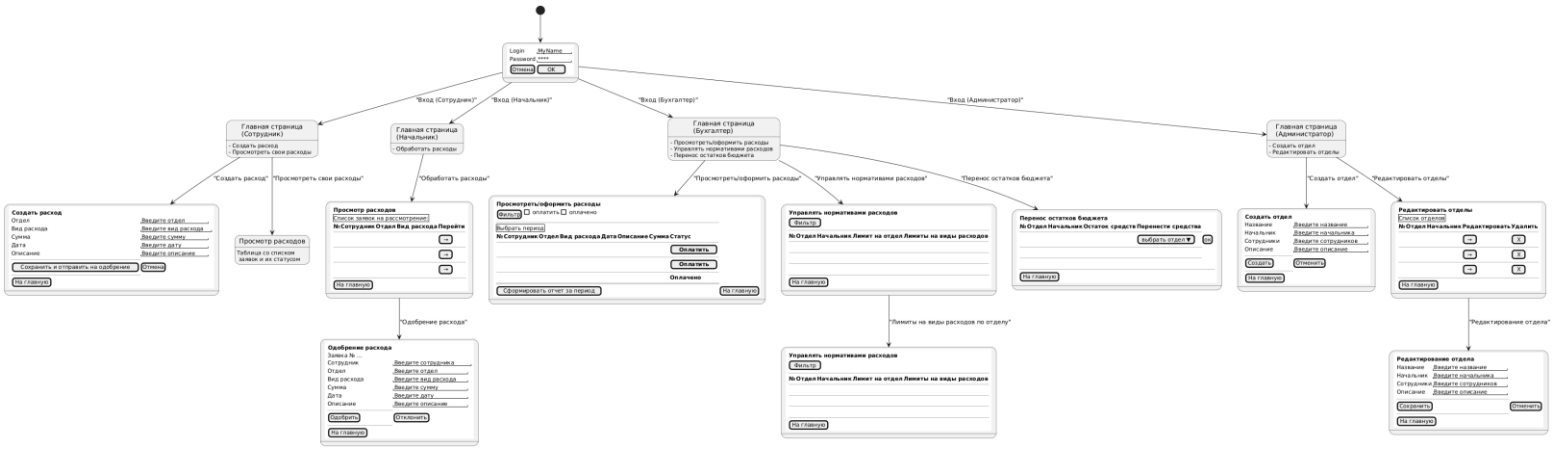

Рисунок 1.9 – Карта диалоговых окон

1.3 Форматы сущностей

На рисунке 1.10 приведена ER-диаграмма базы данных.

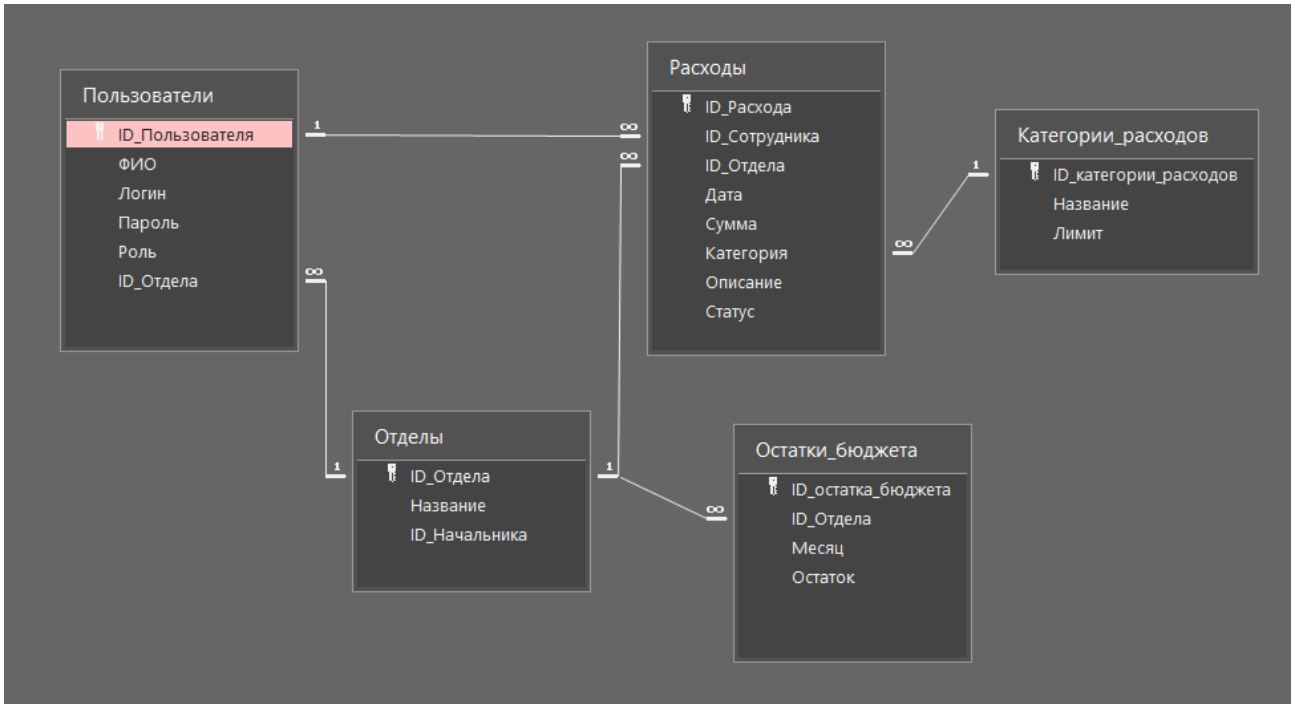

Рисунок 1.10 – ER-диаграмма базы данных

Сотрудник может быть прикреплён только к одному отделу (каждому сотруднику в базе данных привязан один отдел; при создании заявки автоматически определяется его отдел).

При переводе сотрудника из одного отдела в другой, его прошлые расходы остаются привязанными к старому отделу, так как они уже были учтены в бюджете того отдела (т.е. в заявках остается указание на отдел, в котором они были созданы).

Начальник отдела определяется через поле ID_начальника в таблице Отделы. В это поле будет подставляться значение ID_пользователя из таблицы Пользователи. При этом в таблице Пользователи есть отдельное поле Роль, где указываются значения: "администратор", "бухгалтер", "начальник" или "сотрудник". Таким образом, роль "начальник" — это не отдельная сущность, а просто характеристика сотрудника, а его привязка к конкретному отделу осуществляется через ID_начальника в таблице Отделы.

Остатки бюджета для каждого отдела будут добавляться в БД автоматически каждый месяц первого числа в 00:00. Это можно реализовать, например, SQL-запросом INSERT или UPDATE, выполняемым по расписанию через планировщик задач.

Лимиты на виды расходов устанавливаются только бухгалтером через специальный интерфейс системы. Если бухгалтер попытается оформить расход по категории, для которой еще никогда не был установлен лимит (в случае первого запуска системы), система выдаст ошибку и потребует сначала задать лимит. Такой механизм можно реализовать с помощью проверки существования соответствующей записи в таблице.

2 Проектирование

2.1 Прецедент «Создать расход»

На рисунке 2.1 изображена диаграмма пригодности для прецедента «Создать расход».

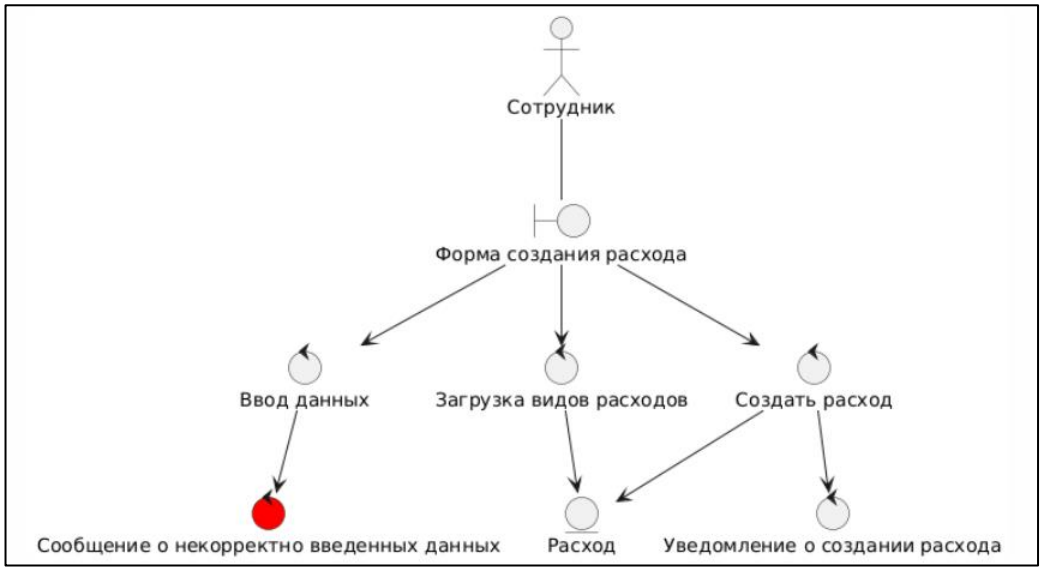

Рисунок 2.1 – Диаграмма пригодности для «Создать расход»

На рисунке 2.2 изображена диаграмма последовательности для прецедента «Создать расход».

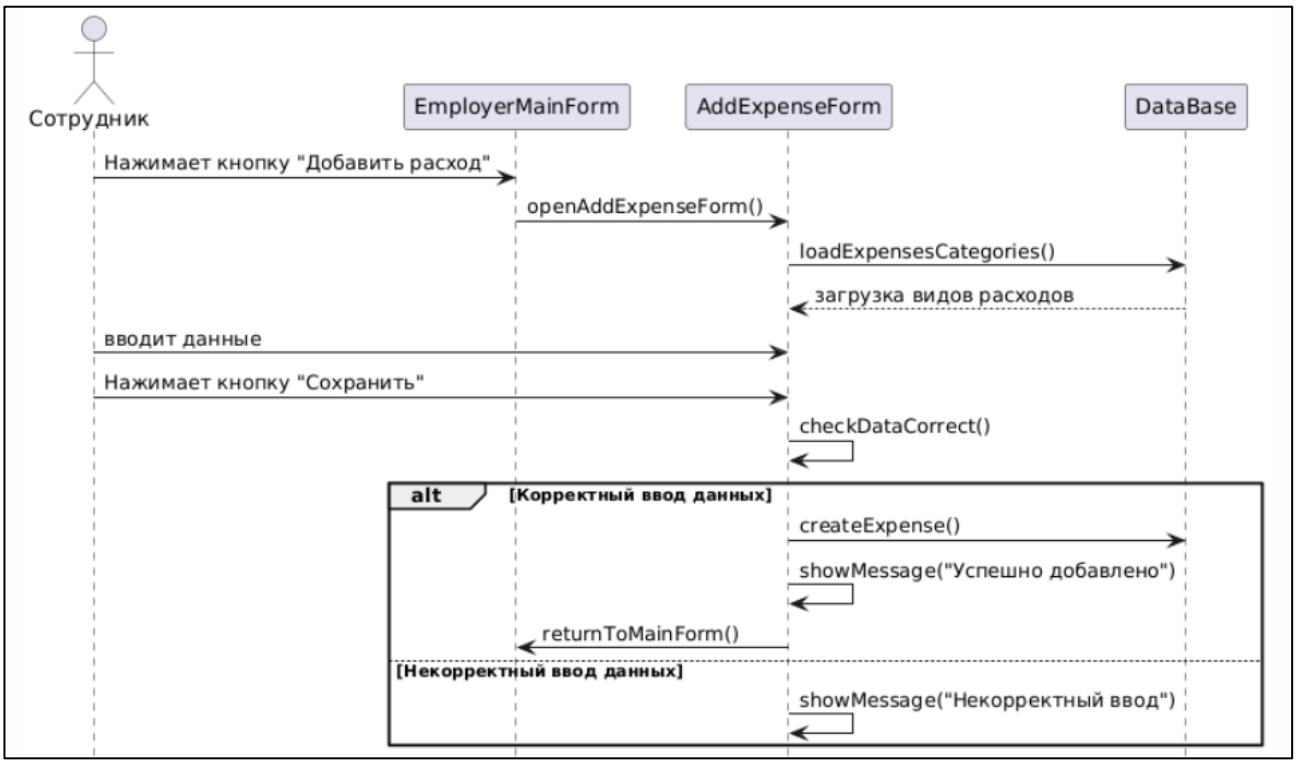

Рисунок 2.2 – Диаграмма последоватльности для «Создать расход»

2.2 Прецедент «Обработать расходы»

На рисунке 2.3 изображена диаграмма пригодности для прецедента «Обработать расходы».

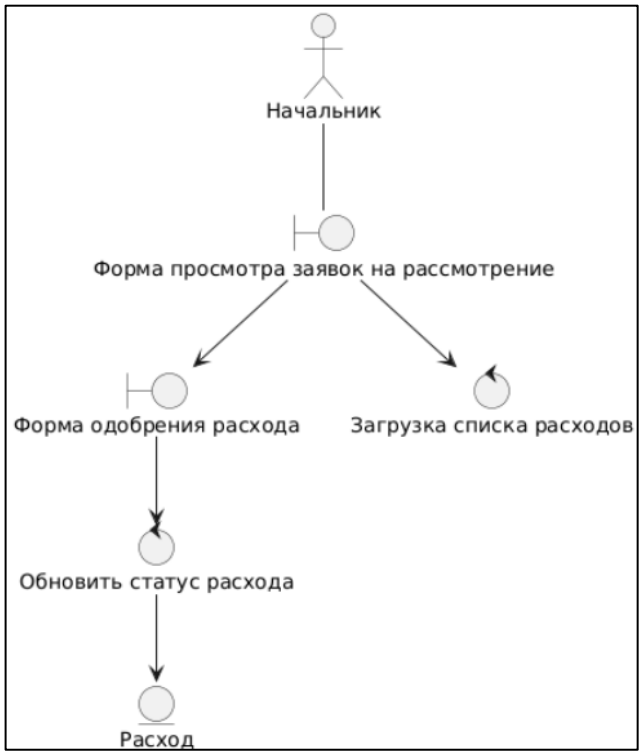

Рисунок 2.3 – Диаграмма пригодности для «Обработать расходы»

На рисунке 2.4 изображена диаграмма последовательности для прецедента «Обработать расходы».

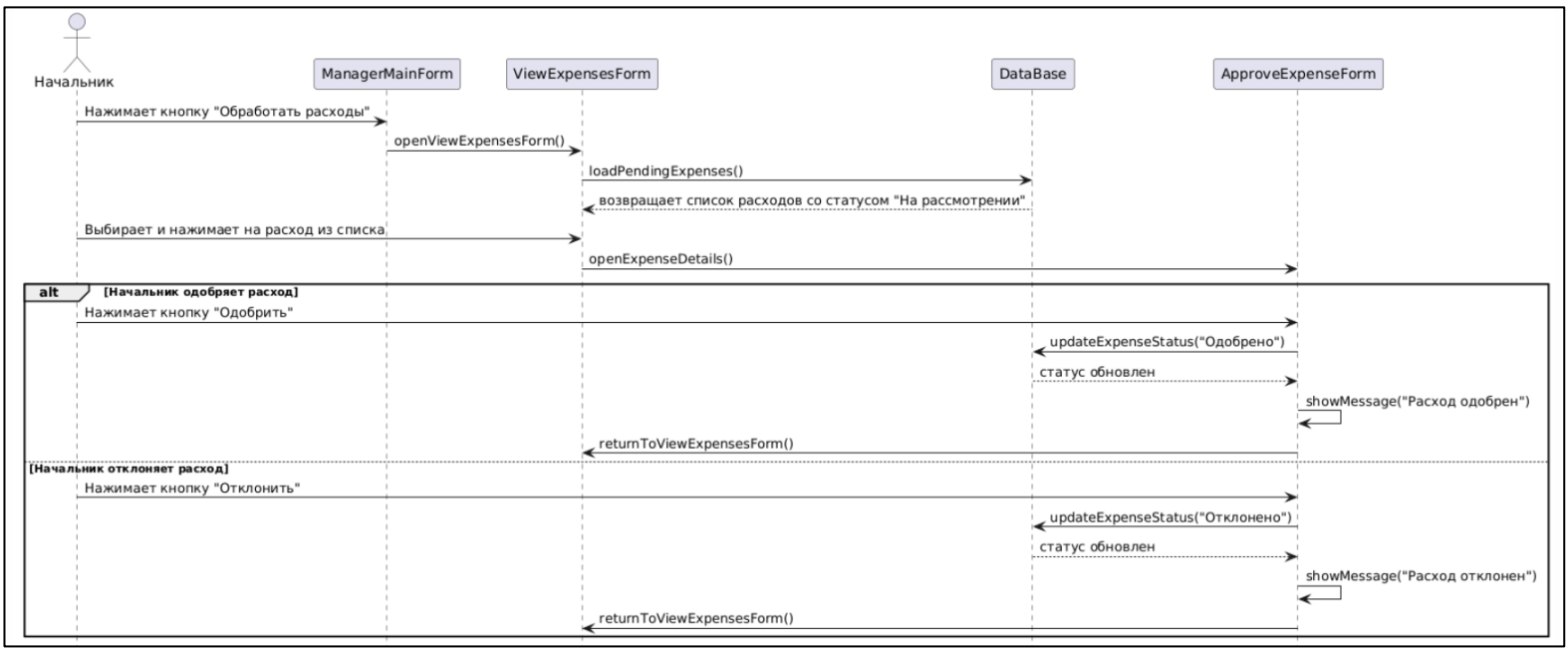

Рисунок 2.4 – Диаграмма последоватльности для «Обработать расходы»

2.3 Прецедент «Оформить расходы»

На рисунке 2.5 изображена диаграмма пригодности для прецедента «Оформить расходы».

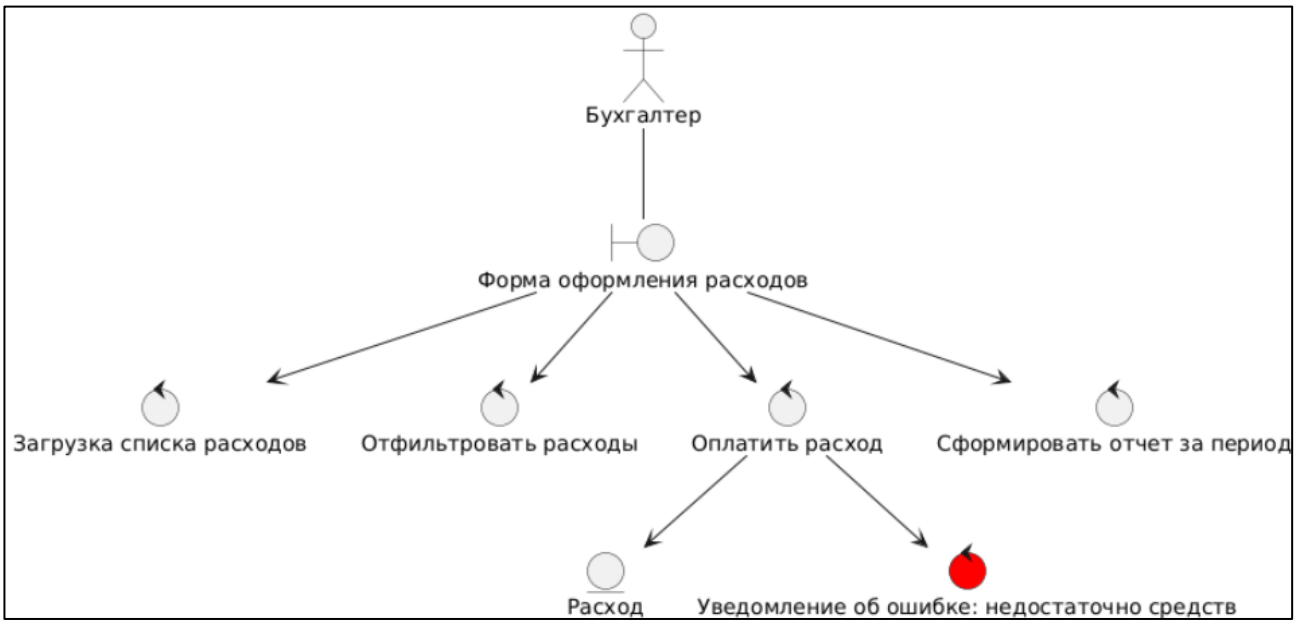

Рисунок 2.5 – Диаграмма пригодности для «Оформить расходы»

На рисунке 2.6 изображена диаграмма последовательности для прецедента «Оформить расходы».

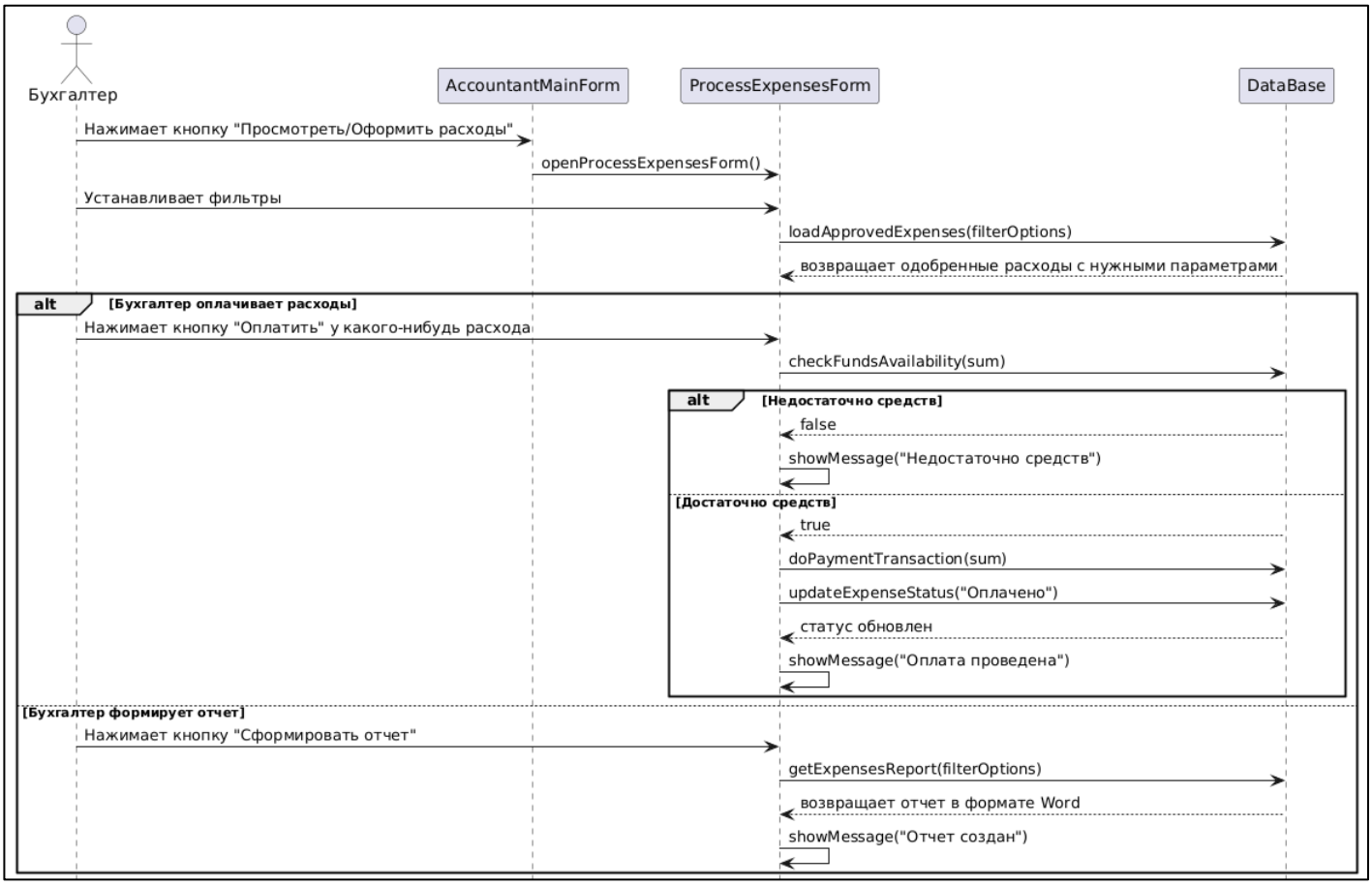

Рисунок 2.6 – Диаграмма последоватльности для «Оформить расходы»

Так как данная ИС делается в рамках упрощенной версии реального проекта, было принято решение не реализовывать отдельно логику проведения банковских транзакций при оплате расходов. То есть метод doPaymentTransaction(sum) здесь является просто с запросом к БД с целью изменить тестовое число, находящееся в поле Остаток в таблице Остатки_бюджета.

2.4 Диаграмма классов

На основе трех диаграмм последовательности по трем ключевым прецедентам была разработана диаграмма классов (рисунок 2.7).

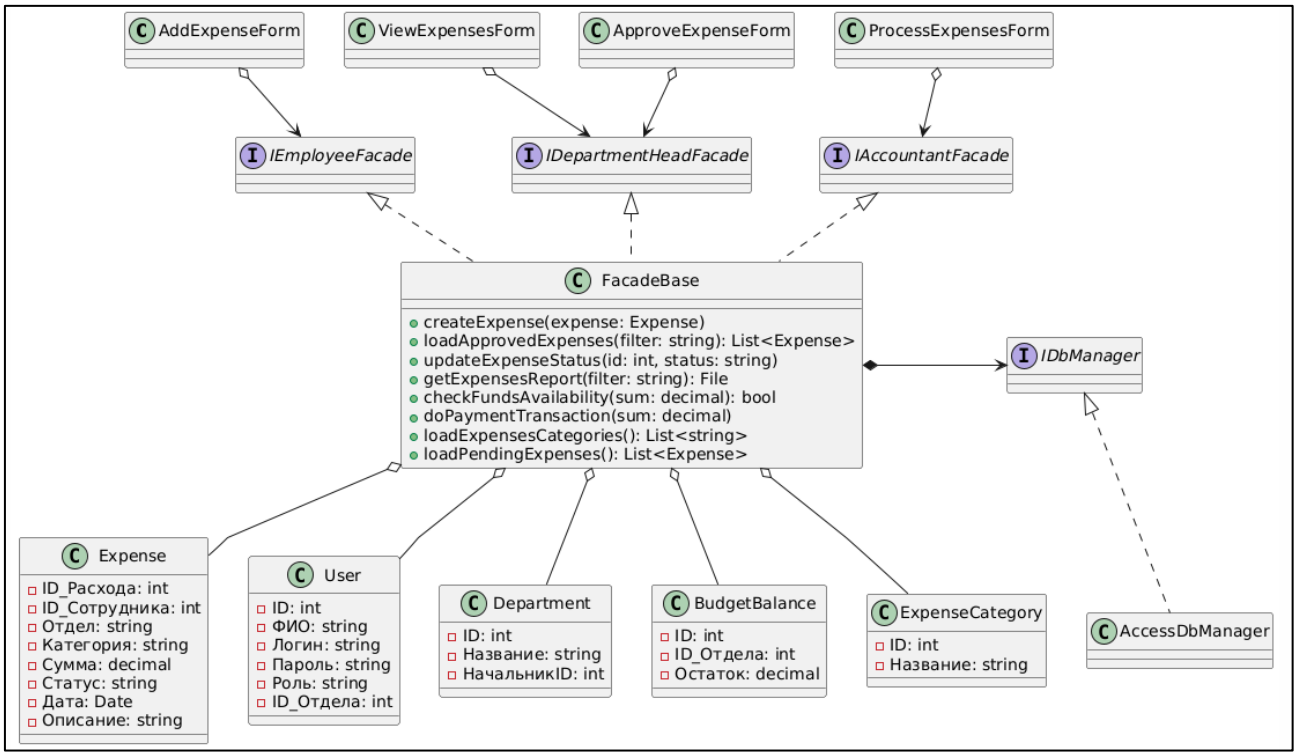

Рисунок 2.7 – Диаграмма классов по трем диаграммам последовательности

IDbManager — интерфейс для работы с базой данных, инкапсулирующий логику доступа к таблицам и изолирующий систему от конкретной реализации хранилища. Такой подход реализует паттерн «Стратегия» и обеспечивает модульность, а также возможность легко заменить СУБД при необходимости. В рамках данной работы используется база данных Access, поэтому реализован класс AccessDbManager, соответствующий интерфейсу IDbManager.

FacadeBase — базовый класс, предоставляющий общие методы для работы с расходами (создание, загрузка, обновление статуса и др.). Реализует паттерн «Фасад», скрывая детали взаимодействия с БД и упрощая работу интерфейсных классов.

IEmployeeFacade, IDepartmentHeadFacade, IAccountantFacade — интерфейсы, которые определяют набор функций для разных ролей пользователей. Каждый интерфейс связывает формы соответствующей роли с базовым фасадом, согласно принципу ISP (Принцип разделения интерфейсов из SOLID).

Модели данных (Expense, Employee и др.) напрямую ассоциированы с FacadeBase, так как фасад использует их для операций с данными.

Каждая форма (например, AddExpenseForm, ProcessExpensesForm) работает через интерфейсы, а не напрямую с БД или моделями, что повышает модульность и облегчает тестирование.

3 Разработка и тестирование

3.1 Работа с данными

Подключение к базе данных Access реализовано с использованием компонента OleDbConnection. Для совместного использования и удобства развертывания база данных ExpensesDatabase.mdb размещается в сетевой папке (SharedDB). Путь к файлу указывается в конфигурационном файле App.config:

```
<connectionStrings>
  <add name="DbConnection"
      connectionString="Provider=Microsoft.Jet.OLEDB.4.0;Data
Source=\\THE_DEATH_STAR\\SharedDB\\ExpensesDatabase.mdb;" />
</connectionStrings>
```

3.2 Учетные данные

Специальные логины или пароли к самой базе данных Access не используются, авторизация реализована на уровне приложения — через таблицу пользователей в базе.

При первом запуске приложения в базе данных уже содержится как минимум одна учётная запись с правами администратора. Это позволяет сразу войти в систему и добавить новых пользователей через интерфейс программы, не редактируя базу вручную.

3.3 Изменение структуры базы данных

Для возможности изменения или замены базы данных применён паттерн «Стратегия». Взаимодействие с СУБД осуществляется через интерфейс IDbManager. Чтобы использовать другую структуру или другую СУБД (например, SQL Server, PostgreSQL), достаточно реализовать новый класс, соответствующий IDbManager, и передать его в FacadeBase. Это позволяет не изменять остальную бизнес-логику системы.

3.4 Сборка программы

Для упрощения запуска приложения был разработан скрипт автоматической сборки build.ps1.

Инструкция по сборке:

1. Клонировать репозиторий:

   git clone https://github.com/AliceNotFromWonderland/expenses-app.git

2. Перейти в каталог проекта и выполнить:

   powershell -ExecutionPolicy Bypass -File build.ps1

   Скрипт (приложение А):

   – проверяет наличие .NET Framework 4.7.2 и при необходимости предлагает установить его,

   – проверяет наличие базы данных по сетевому пути,

   – собирает решение через MSBuild.

После успешной сборки исполняемый файл Uchet_Расходов.exe будет находиться прямо в корневой папке.

3.5 Тестирование

Для модульного тестирования в проекте создан подпроект UnitTestProject с использованием фреймворка MSTest. Для оценки покрытия кода применялись утилиты OpenCover и ReportGenerator, с помощью которых был сформирован HTML-отчёт, представленный на рисунке 3.1.

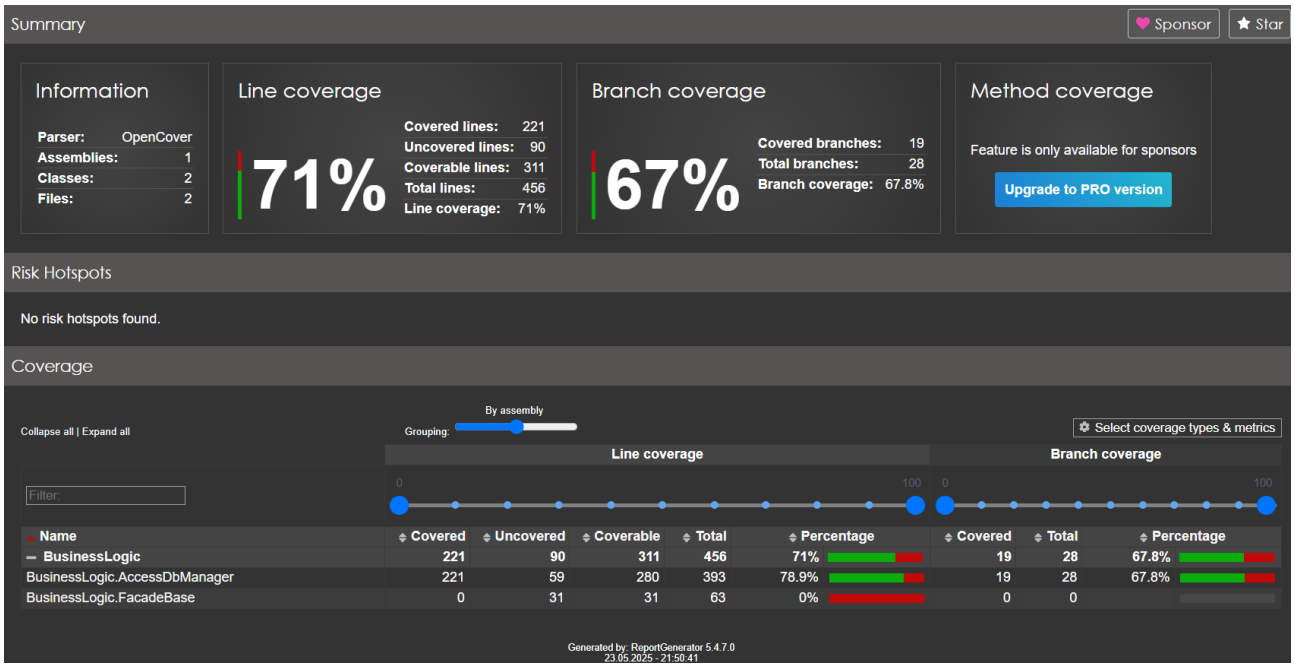

Рисунок 3.1 – Процент покрытия кода тестами

ЗАКЛЮЧЕНИЕ

В рамках данной курсовой работы была спроектирована и реализована информационная система «Учет внутриофисных расходов», предназначенная для автоматизации финансовых процессов внутри организации. Разработанное Windows Forms-приложение обеспечивает удобный интерфейс для взаимодействия с базой данных и поддерживает работу различных ролей: администратора, начальника, бухгалтера и сотрудника. Система позволяет фиксировать расходы, подавать и согласовывать заявки, контролировать оплату, а также формировать отчеты за указанные периоды. Работа подтверждает получение навыков проектирования, создания модульных тестов, работу с базами данных, работы с ООП.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Создание проекта форм интерфейса и карты диалоговых окон в PLANTUML [Электронный ресурс]. – Режим доступа: https://habr.com/ru/articles/279373/ (дата обращения 23.05.2025)

2. Создание базы данных в Access [Электронный ресурс]. – Режим доступа: https://support.microsoft.com/ru (дата обращения 23.05.2025)

3. Процесс ICONIX. Диаграммы пригодности [Электронный ресурс]. – https://pro-prof.com/archives/2723 (дата обращения 23.05.2025)

4. Нотации модели сущность-связь (ER диаграммы) [Электронный ресурс]. – https://pro-prof.com/archives/8126 (дата обращения 10.05.2025)

5. Metanit – Facade Pattern [Электронный ресурс]. – Режим доступа: https://metanit.com/sharp/patterns/4.3.php. (дата обращения: 23.05.2025)

6. MSTest [Электронный ресурс]. – Режим доступа: https://www.nuget.org/packages/MSTest.TestFramework (дата обращения 23.05.2025)

7. OpenCover [Электронный ресурс]. – Режим доступа: https://github.com/OpenCover/opencover/releases (дата обращения 23.05.2025)

8. ReportGenerator [Электронный ресурс]. – Режим доступа: https://github.com/danielpalme/ReportGenerator/releases (дата обращения 23.05.2025)

9. Репозиторий GitHub [Электронный ресурс]. – Режим доступа: https://github.com/AliceNotFromWonderland/expenses-app.git (дата обращения 23.05.2025)

ПРИЛОЖЕНИЕ А

Скрипт автоматической сборки build.ps1 для запуска приложения

```
[Console]::OutputEncoding = [System.Text.Encoding]::UTF8
Write-Host "Проверка наличия .NET Framework 4.7.2..."
$regPath = "HKLM:\SOFTWARE\Microsoft\NET Framework Setup\NDP\v4\Full"
$version = (Get-ItemProperty -Path $regPath -Name Release -ErrorAction SilentlyContinue).Release
if ($version -lt 461808) {
    Write-Warning "Не установлен .NET Framework 4.7.2. Установка через winget..."
    winget install Microsoft.DotNet.Framework.DeveloperPack_4
    exit 1
} else {
    Write-Host ".NET Framework 4.7.2 установлен."
}
# Проверка наличия базы данных
$dbPath = "\\THE_DEATH_STAR\SharedDB\ExpensesDatabase2.mdb"
if (-Not (Test-Path $dbPath)) {
    Write-Warning "Файл базы данных не найден по пути: $dbPath"
    exit 1
} else {
    Write-Host "База данных найдена: $dbPath"
}
# Сборка проекта
Write-Host "Сборка проекта..."
$msbuildPath = "C:\Program Files\Microsoft Visual Studio\2022\Community\MSBuild\Current\Bin\MSBuild.exe"
& $msbuildPath .\UchetRashodov.sln /p:Configuration=Release
```

```
if ($LASTEXITCODE -ne 0) {
    Write-Error "Ошибка при сборке."
    exit 1
} else {
    Write-Host "Сборка завершена успешно."
}
```